



INSTITUT FOR DATALOGI
AALBORG UNIVERSITET

Imperativ programmering

Lektion 6: Mere om funktioner

Pointere anvendt som outputparametre

Slides lavet af Morten Schou, Peter G. Jensen, Anders Schlichtkrull, Kurt Nørmark; Institut for Datalogi, Aalborg Universitet

Læringsmål

Opnå viden og færdigheder om

- Det basale pointerbegreb
  - Adresse og dereferencing operatorerne
- Output-parametre til funktioner
- Funktioner som parameter
  - Funktionspointere

} Fokus i dagens live coding.

Dagsplan

- Live coding med funktioner med output-parametre
  - Et simpelt eksempel
  - Et eksempel med **`scanf`**
  - Et eksempel med deference-operatoten

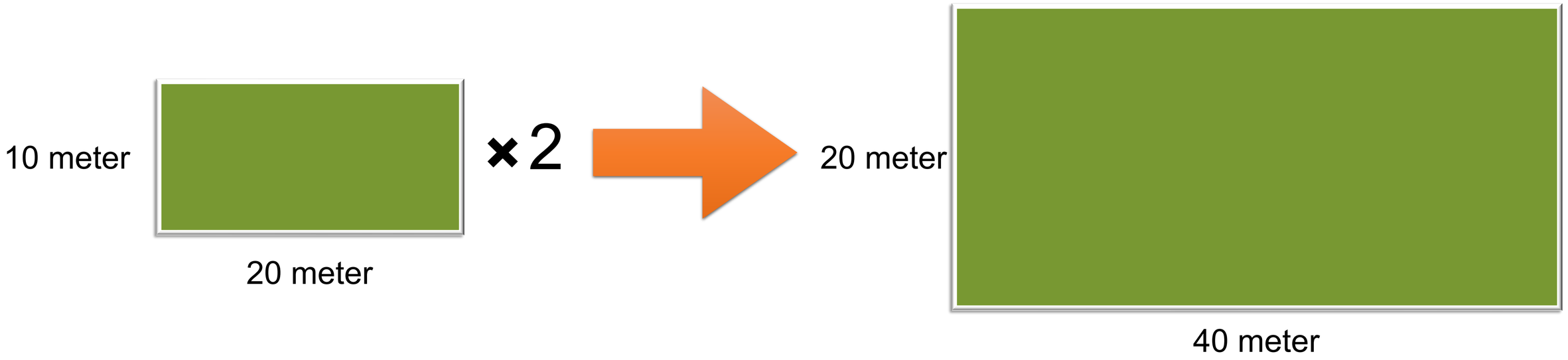
- Tommelfingerregel for output
- Om Miniprojekt 1

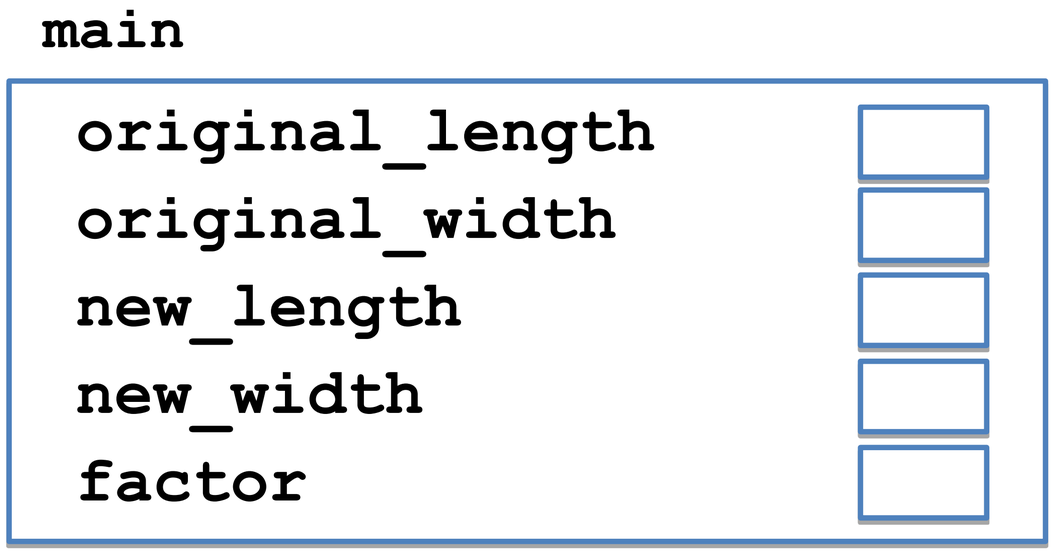
- Gruppeopgaver

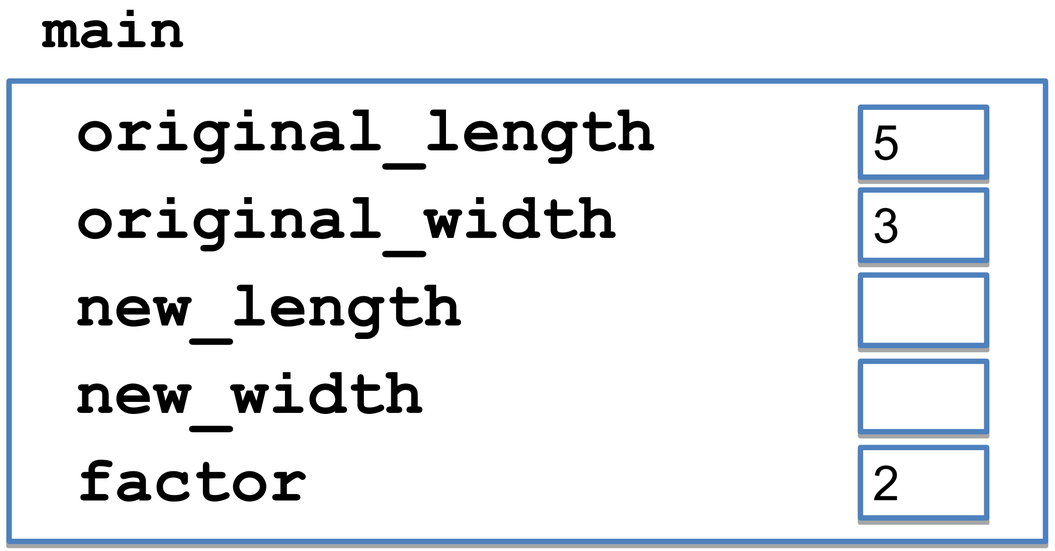
Græsplæneudvidelse

Programmet tager som input længden og bredden af en græsplæne (i meter). Programmet tager også som input en forstørrelsesfaktor *n*.

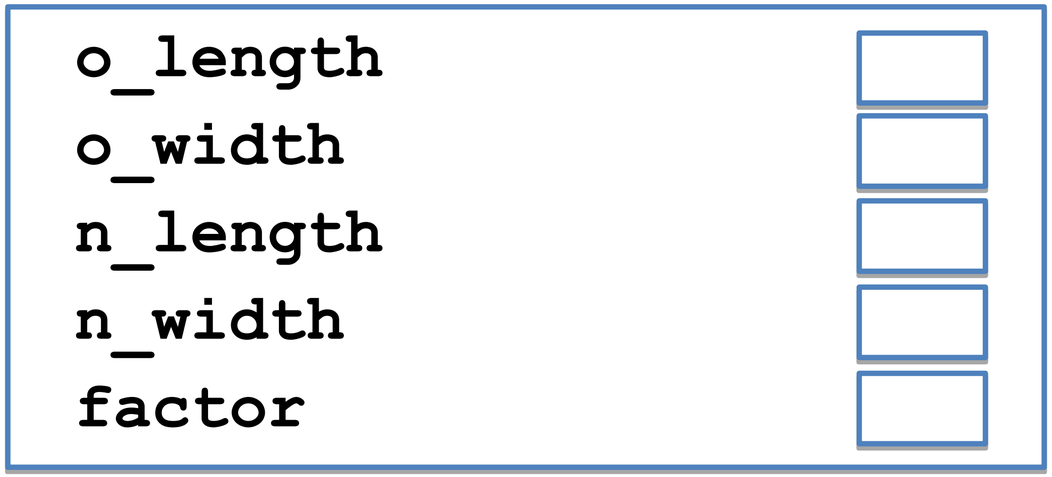
Programmet udregner vha en funktion længde og bredde for græsplænen, hvis dens sider blev forstørret med faktor *n*.



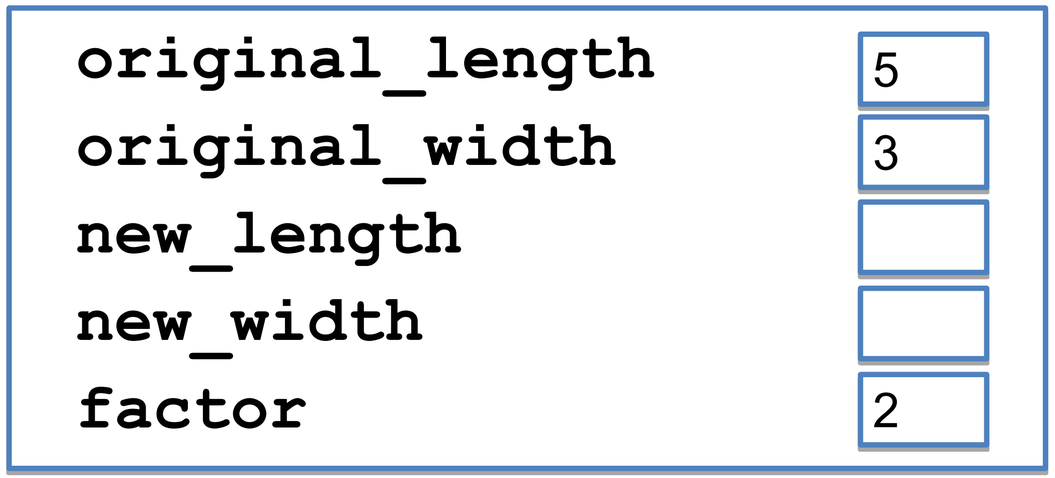

main
original_length
original_width
new_length
new_width
factor

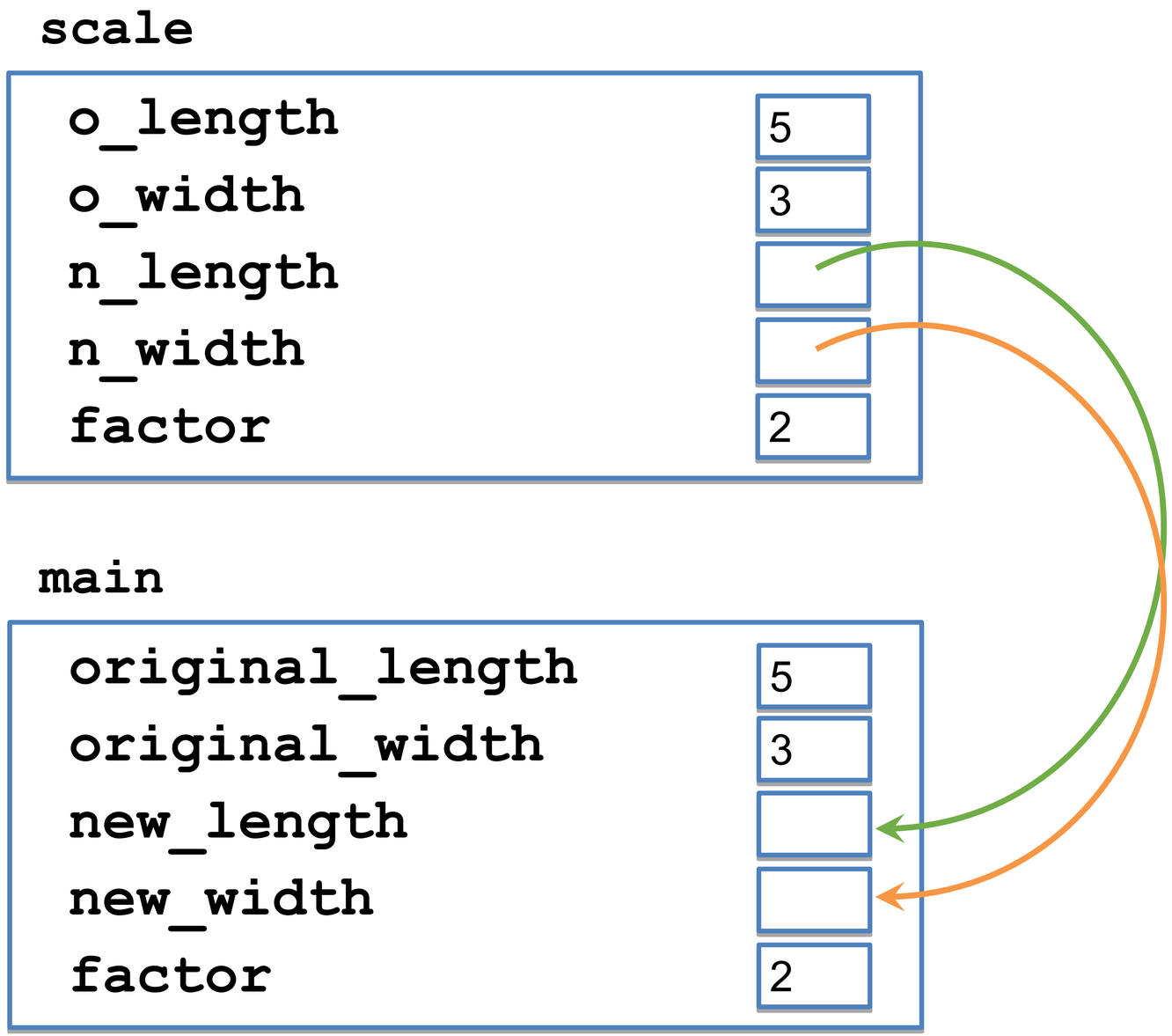

main
original_length
5
original_width
3
new_length
new_width
factor
2

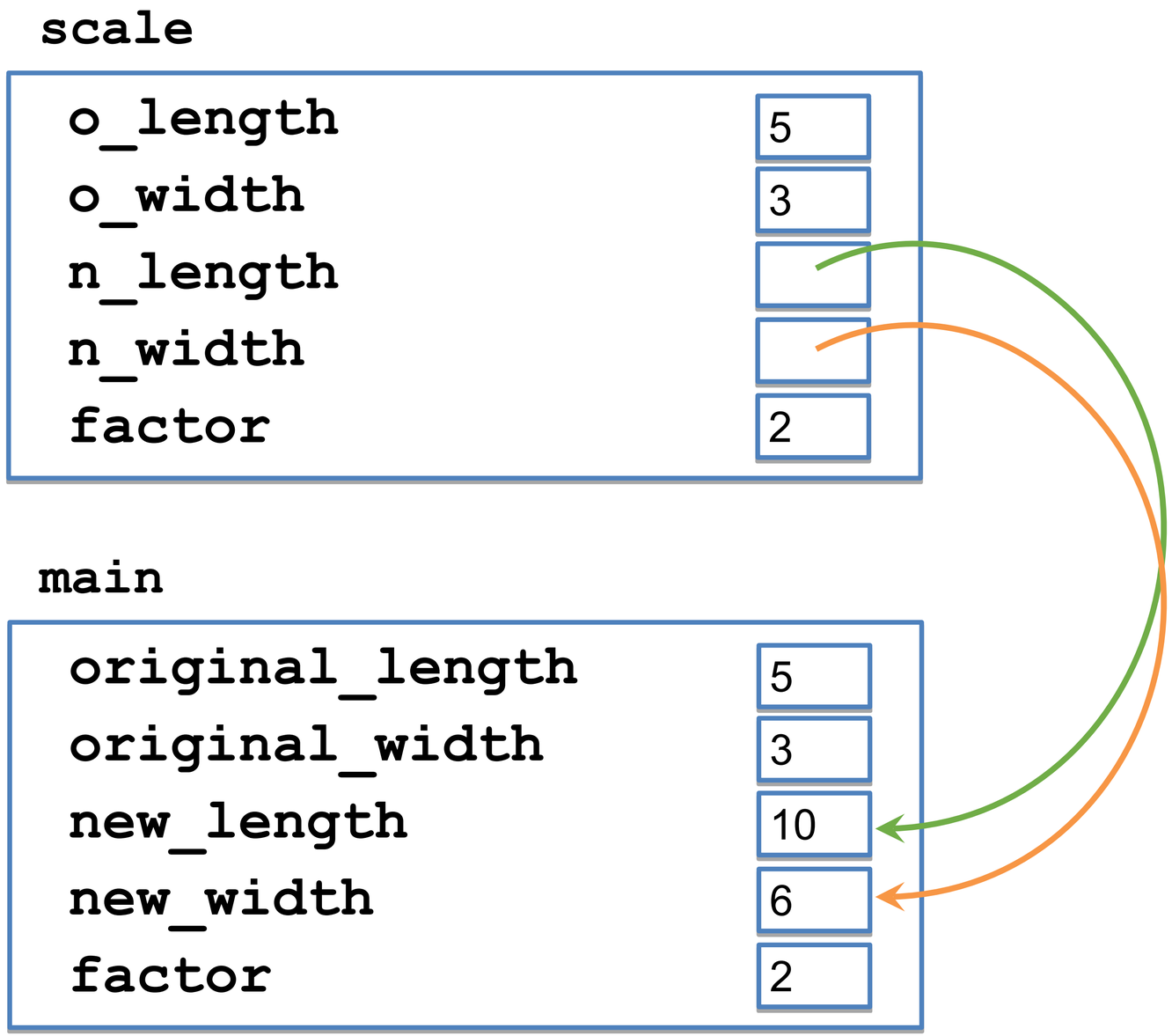
scale

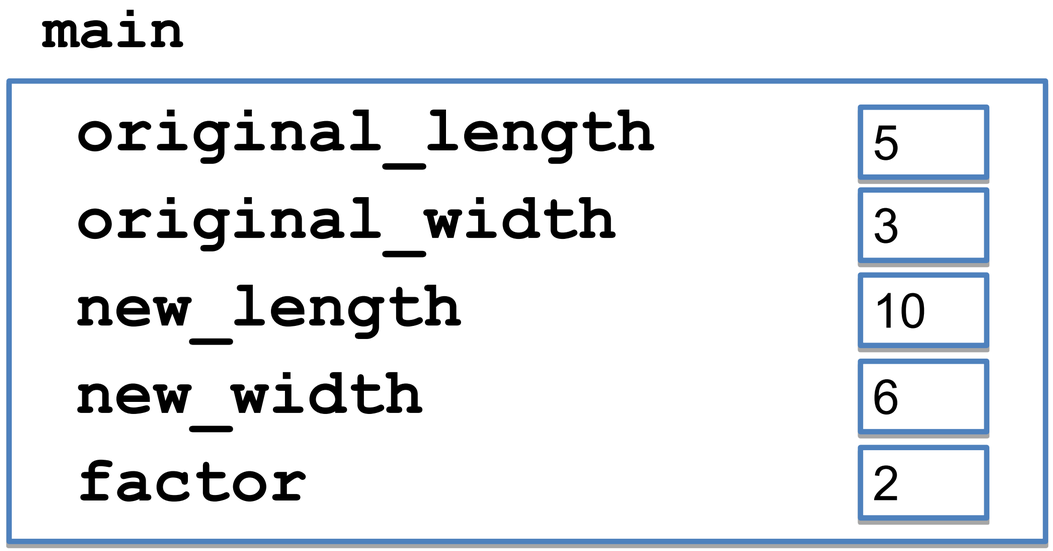

o_length
o_width
n_length
n_width
factor

main


original_length
5
original_width
3
new_length
new_width
factor
2


scale
o_length
o_width
n_length
n_width
factor
5
3
2
main
original_length
original_width
new_length
new_width
factor
5
3
2


scale
o_length
o_width
n_length
n_width
factor
5
3
2
main
original_length
original_width
new_length
new_width
factor
5
3
10
6
2


main
original_length
5
original_width
3
new_length
10
new_width
6
factor
2

Auditorieopgave A

- I løser opgaven med jeres sidemakker.

- I har 15 minutter.
- Løser man opgaven inden de 15 minutter har man pause.

- Der er også en "bonus opgave".

Auditorieopgave A – Vektor addition

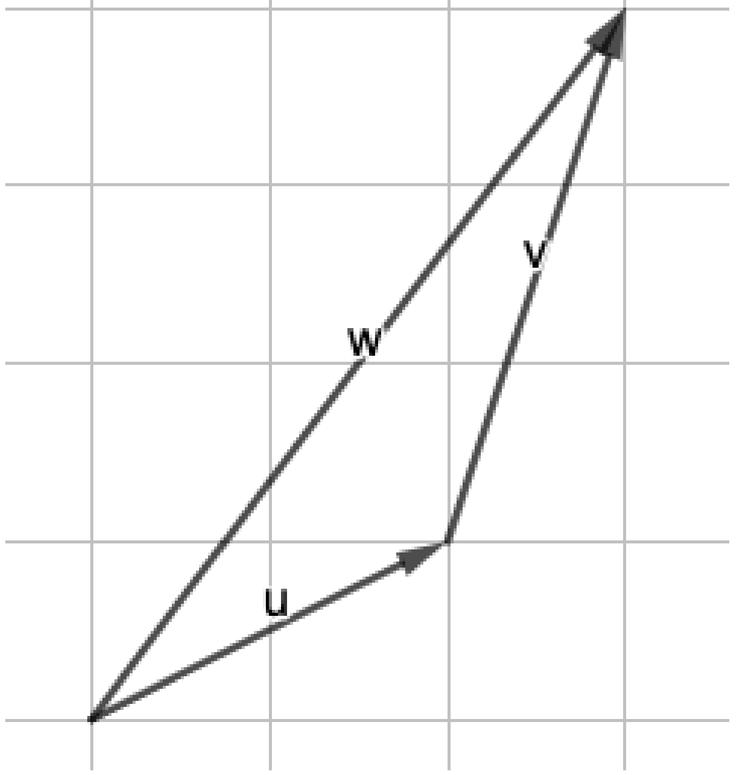
(Bonus opgave:
Lav "gange med skalar", "minus", "krydsprodukt" og "længde".)

- Formel

$$\begin{pmatrix} u_1 \\ u_2 \end{pmatrix} + \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} u_1 + v_1 \\ u_2 + v_2 \end{pmatrix}$$

- Eksempel

$$\begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 3 \end{pmatrix} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$$



- I skal skrive et program som kan lægge to vektorer sammen.
- Lav en funktion **`add`** med:
  input-parametre svarende til u_1, u_2, v_1, v_2
  output-parametre svarende til w_1 w_2 (resultat koordinaterne).

Hvis det ikke er relateret til elefanter, er det irrelefant!



Græsplæne opgave

- I græsplæne opgaven ønsker vi nu at brugeren skal indtaste længde, bredde og ”forstørrelses-faktoren”.
- Vi ønsker at en funktion `get_input` tager sig af dette.

Auditorieopgave B

- I løser opgaven med jeres sidemakker.

- I har 15 minutter.
- Løser man opgaven inden de 15 minutter har man pause.

- Der er også en "bonus opgave".

Auditorieopgave B – Trekantsopgaven

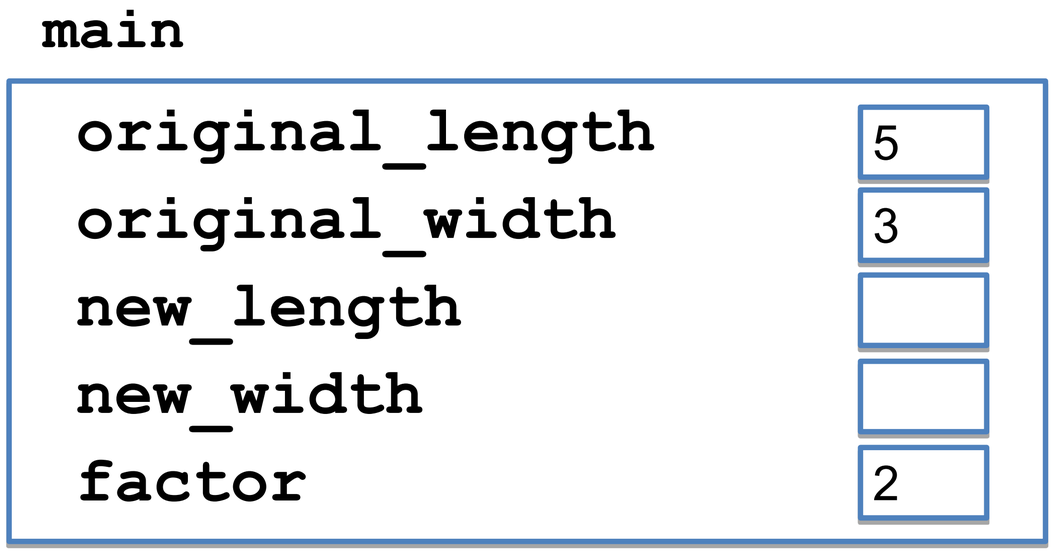
- Vi kigger på trekantsopgaven fra sidst.

- Indfør en funktion `get_input` som beder brugeren om input og lægger det indtastede input ind i main-funktionens variable (`p1_x, p1_y, p2_x, p2_y, p3_x, p3_y`).
  - Gør dette ved brug af passende output-parametre.

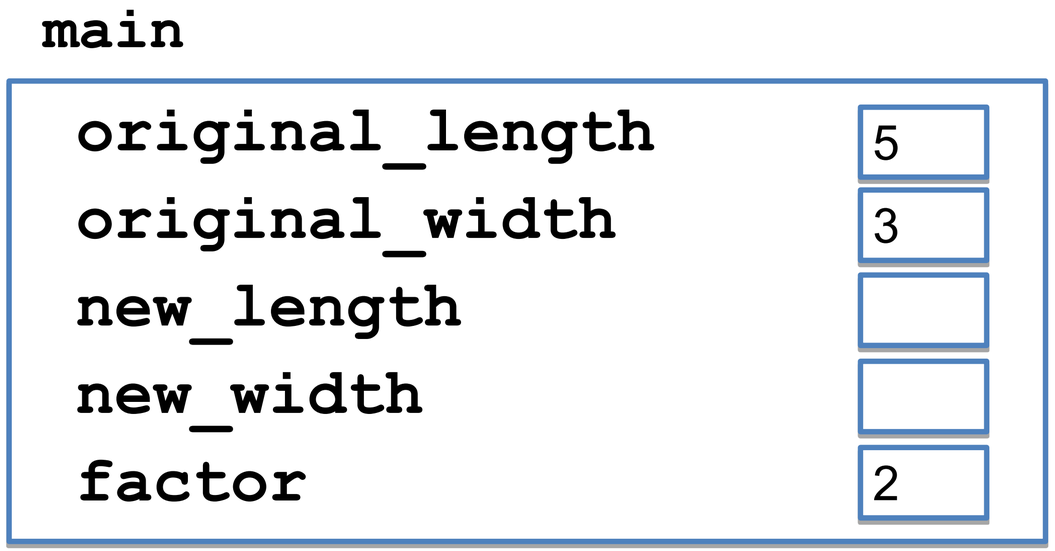
(Bonus: Modificer `compute_sum_of_length` sådan at den giver 4 outputs, nemlig længderne af de tre sider samt den samlede længde. Gør dette vha. passende output-parametre.)

Opsamling

Græsplæneudvidelse – igen



Græsplæneudvidelse – igen



Vi kører nu vores funktion

Græsplæneudvidelse – igen

main

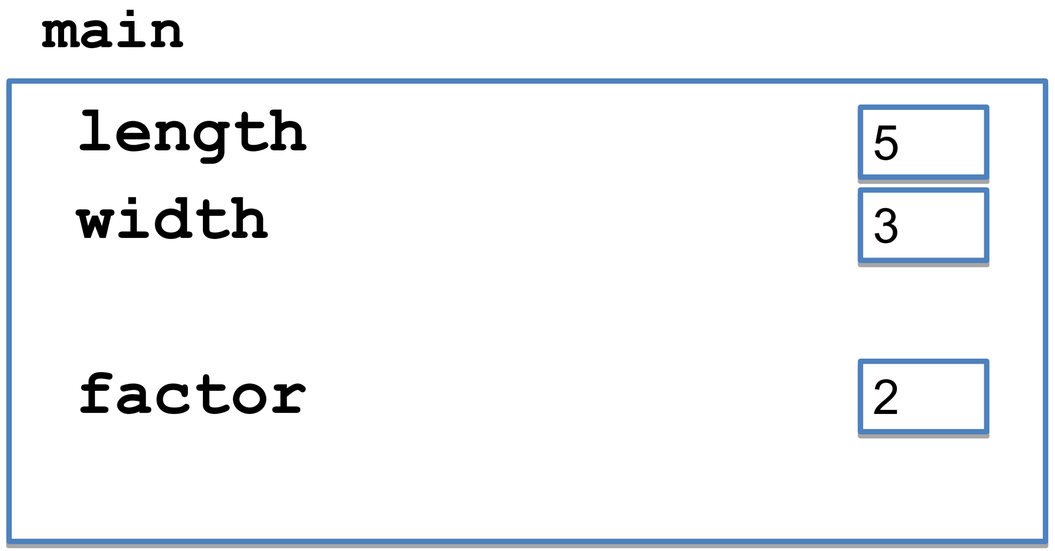
`original_length`	5
`original_width`	3
`new_length`	10
`new_width`	6
`factor`	2

Vi kører nu vores funktion

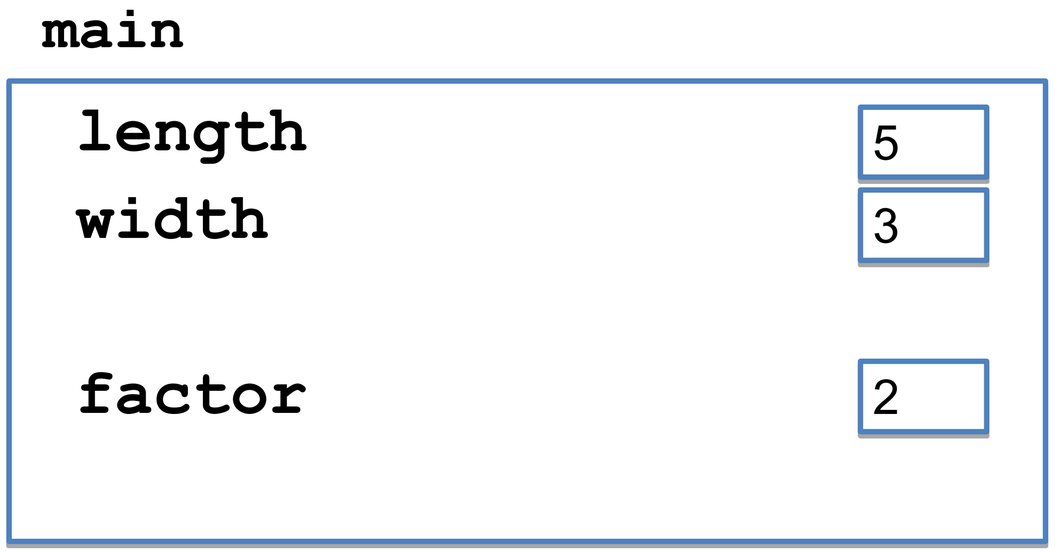
Græsplæneudvidelse – igen

Lad os se på et alternativ hvor vi kun har ét sæt `length` og `width` variable

Græsplæneudvidelse – igen

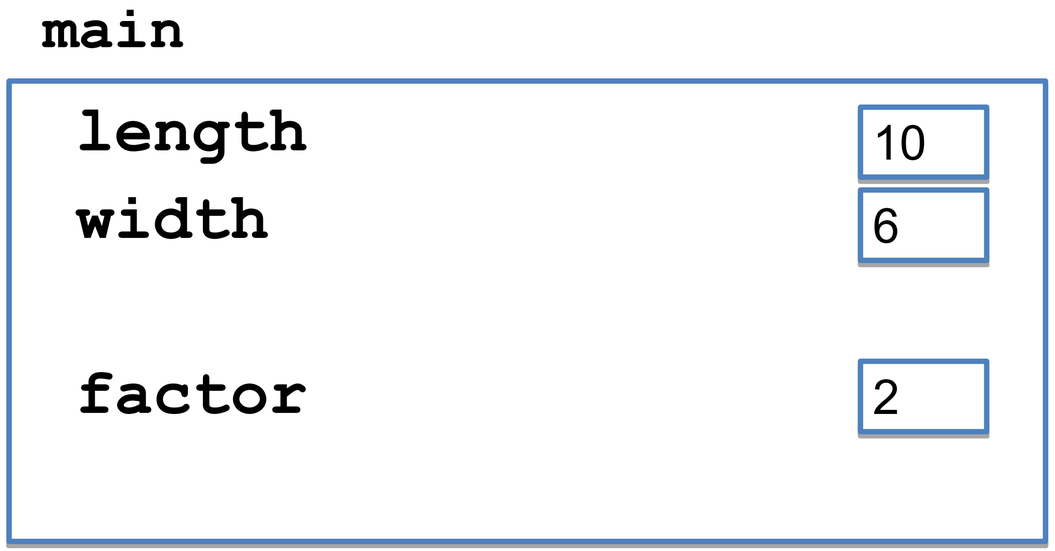


Græsplæneudvidelse – igen



Vi kører nu vores funktion

Græsplæneudvidelse – igen



Vi kører nu vores funktion

Græsplæne opgave – igen

- Jeg programmerer græsplæneopgaven så den virker som illustreret.
- Tricket er at have parametre som både er input og output-parametre.

Tommelfinger-regel

- Hvis din funktion har ét output:
  - Brug return

- Hvis din funktion har flere output:
  - Brug output-parametre
  - (eller structs – men det gemmer vi til en senere lektion)

Om Miniprojekt 1

Miniprojekt 1

- Opgavebeskrivelsen ligger på Moodle / CodeGrade.
- Men I får også en forklaring nu.

Miniprojekt 1

En regnemaskine på kommandolinjen

Akkumulator: 0

Miniprojekt 1

En regnemaskine på kommandolinjen



Miniprojekt 1

En regnemaskine på kommandolinjen



Miniprojekt 1

En regnemaskine på kommandolinjen

#

Akkumulator: 5

```
#     for kvadratroden af akkumulatoren
%     for at vende fortegnet af akkumulatoren
!     for at dividere 1 med akkumulatoren
q     for at afslutte regnemaskinen med slutresultatet
```

Miniprojekt 1

En regnemaskine på kommandolinjen




```
#     for kvadratroden af akkumulatoren
%     for at vende fortegnet af akkumulatoren
!     for at dividere 1 med akkumulatoren
q     for at afslutte regnemaskinen med slutresultatet
```

Miniprojekt 1

En regnemaskine på kommandolinjen




```
#     for kvadratroden af akkumulatoren
%     for at vende fortegnet af akkumulatoren
!     for at dividere 1 med akkumulatoren
q     for at afslutte regnemaskinen med slutresultatet
```

Miniprojekt 1

En regnemaskine på kommandolinjen

```
                                  q
Akkumulator:  -8
```

Resultatet er -8

```
#     for kvadratroden af akkumulatoren
%     for at vende fortegnet af akkumulatoren
!     for at dividere 1 med akkumulatoren
q     for at afslutte regnemaskinen med slutresultatet
```

Ønskede funktioner

- **`run_calculator`**
  - Abstraherer og indkapsler dialogen i den samlede regnemaskine
- **`scan_data`**
  - Abstraherer indlæsning
  - Skal indlæse en operator
  - Skal også indlæse en operand hvis operatoren er binær
  - To output parametre
- **`do_next_op`**
  - Input: operator og operand
  - Input/output: akkumulatoren
- En boolsk funktion der afgør om en operator er binær
  - Input: Operatoren

Unit testing

- Jeg lægger op til at I skriver unit test af funktionen **`do_next_op`**
- Disse test skrives i filen **`test/test_calculator.c`** hvor jeg også har givet et eksempel på en unit test.

- Hvis det bliver for meget at nå, så er det vigtigste at I får arbejdet med at programmere selve regnemaskinen. I kan kigge mere på unit testing senere.

Aflevering

- Deadline: 27.10.2026 kl. 21.00
- Aflevér på CodeGrade – enten i CodeGrade's editor eller gennem Github.

- Du kan som sædvanligt aflevere/genaflevere så tit du ønsker det frem mod afleveringsfristen.

Gruppeopgaver

- Vi går nu til grupperummet for at lave opgaveopgaver
- I sætter jer selvfølgelig i jeres grupper.
- Husk at sørge for at alle er med før I går videre til næste opgave.



INSTITUT FOR DATALOGI

AALBORG UNIVERSITET